

Latency Percentiles & Tail at Scale, Worked Out by Hand

The mean hides the tail. We compute p50/p90/p99 from a real sample, show why the average misleads, and derive the “tail at scale” that makes fan-out slow.

What this document does. “How slow is the server?” has no single answer — latency is a distribution. This document computes p50, p90, and p99 from a concrete sample, contrasts them with the misleading mean, defines the error rate against an SLO, and derives the tail-at-scale effect: why a request that touches many servers is almost always limited by the slowest one.

Concepts covered: percentiles by nearest-rank · p50/p90/p99 · mean vs median under skew · error rate and SLO · tail amplification $(1 - q)^N$.

Internal learning material. Numbers reproducible from the appendix code. v1.0

1 The setup: ten measured latencies

The client records the round-trip latency (ms) of $n = 10$ recent tool calls. Sorted ascending:

$$x = [20, 22, 25, 28, 30, 35, 42, 55, 120, 800].$$

Nine calls are fast; one (800 ms) is a tail event — a GC pause, a cold cache, a retried packet. That single outlier is the whole story.

2 Step 1 — Percentiles by nearest-rank

The P -th percentile (nearest-rank method) sits at sorted position

$$\text{rank} = \lceil \frac{P}{100} n \rceil, \quad p_P = x_{[\text{rank}]}$$

With $n = 10$:

$$p50 : [0.50 \cdot 10] = 5 \Rightarrow x_{[5]} = 30, \quad p90 : [0.90 \cdot 10] = 9 \Rightarrow x_{[9]} = 120, \quad p99 : [0.99 \cdot 10] = 10 \Rightarrow x_{[10]} = 800.$$

statistic	value (ms)	meaning
p50 (median)	30	half of calls are faster
p90	120	1 in 10 is slower
p99	800	the tail a user actually feels
mean	117.7	arithmetic average

3 Step 2 — Why the mean lies

The arithmetic mean is

$$\bar{x} = \frac{20 + 22 + 25 + 28 + 30 + 35 + 42 + 55 + 120 + 800}{10} = \frac{1177}{10} = 117.7 \text{ ms.}$$

But $\bar{x} = 117.7$ is *slower than 90% of all calls* — it sits above even p90. A single 800 ms outlier dragged the average far from the typical experience (30 ms). The median is robust to that outlier; the mean is not.

Intuition

Latency distributions are right-skewed: a floor (you can't be faster than the network) and a long tail (anything can go wrong). The mean chases the tail; the median reports the typical call. Report *percentiles*, not averages — p50 for “what most users see,” p99 for “what your worst users see.” An SLA written on the mean is gameable and meaningless.

4 Step 3 — Error rate against an SLO

Define an SLO: “a call is good if it succeeds and returns under 200 ms.” In the sample, 9 of 10 are under 200 ms and (say) all succeeded, so

$$\text{good rate} = \frac{9}{10} = 0.90, \quad \text{SLO violation rate} = 0.10.$$

A 99.9% latency SLO would be badly missed here — the 800 ms call alone is a 10% violation in this window.

5 Step 4 — The tail at scale

Here is the consequence that makes the tail dominate. Suppose each server keeps p99 good — i.e. each call is “fast” with probability $q = 0.99$ (slow 1% of the time). A single user request fans out to N servers and waits for *all* of them (Module 3.1's max). The probability the whole request is fast is

$$P(\text{all fast}) = q^N, \quad P(\text{at least one slow}) = 1 - q^N.$$

N	1	10	100
$q^N = 0.99^N$	0.990	0.904	0.366
$1 - q^N$ (slow)	1.0%	9.6%	63.4%

At $N = 100$ fan-out, a request hits at least one slow server 63% of the time — even though every *individual* server is 99% fast. The system's p99 becomes the user's median.

Watch out

This is why “our servers are 99% fast” does not mean “users are 99% happy.” The more servers a request touches, the more the rare tail dominates. Fixes attack N or the tail directly: hedge slow calls (send a duplicate after p95), cut fan-out, or tighten each server's p99 — because q^N punishes scale exponentially.

6 The argument, at a glance

#	Idea	Formula	Value
1	Percentile rank	$\lceil \frac{P}{100} n \rceil$	—
1	p50 / p90 / p99	sorted lookup	30 / 120 / 800
2	Mean (skewed)	$\frac{1}{n} \sum x_i$	117.7
3	SLO violation rate	bad/total	0.10
4	Tail at scale	$1 - q^N$	63.4% @ $N=100$

7 Equation cheat-sheet

Percentile rank	$\lceil (P/100) n \rceil$
Percentile value	$p_P = x_{[\text{rank}]}$ (sorted)
Mean	$\bar{x} = \frac{1}{n} \sum_i x_i$
SLO violation rate	(calls breaching SLO) / total
All-fast probability	q^N (fan-out N , each fast w.p. q)
Tail at scale	$1 - q^N$

A Reference implementation

```
import math
x = sorted([20,22,25,28,30,35,42,55,120,800])
n = len(x)
def pct(P):
    return x[math.ceil(P/100 * n) - 1]
print(pct(50), pct(90), pct(99))           # 30 120 800
print(sum(x)/n)                           # 117.7 (mean)

q = 0.99
for N in (1, 10, 100):
    print(N, round(1 - q**N, 3))           # 0.01 0.096 0.634
```

— end of document —